# PaperWings: A Tutorial on Vector Symbolic Architectures for Associative Memory and Knowledge Representation

Seamus Brady
Independent Researcher
Dublin, Ireland
seamus@corvideon.ie
https://seamusbrady.ie

**Abstract**

Vector Symbolic Architectures (VSAs), also known as Hyperdimensional Computing (HDC), offer a brain-inspired computational framework in which data is represented as high-dimensional random vectors and manipulated through simple algebraic operations. Despite growing interest, accessible and pedagogical implementations remain limited. We present *PaperWings*, a lightweight Python library that implements three VSA vector types (binary, bipolar, and binary sparse), provides an associative memory with exponential decay, and introduces a triple-unbinding algorithm for knowledge graph recovery. This paper provides background on the VSA framework, walks through the library's design and API, and demonstrates its use in encoding semantic relationships, performing associative recall, and recovering knowledge graph triples.

**Keywords:** vector symbolic architectures, hyperdimensional computing, associative memory, knowledge representation, Python

## 1 Introduction

Representing structured knowledge in a way that supports robust, noise-tolerant retrieval is a long-standing challenge in artificial intelligence. Conventional symbolic systems offer compositionality but are brittle; neural networks offer robustness but struggle with explicit relational structure. Vector Symbolic Architectures (VSAs) occupy a middle ground: they encode symbols as high-dimensional random vectors and compose them via algebraic operations that preserve the ability to decompose and query the resulting representations [Gayler, 2003, Kanerva, 2009].

Interest in VSAs has accelerated in recent years, driven by applications in classification [Rahimi et al., 2016, Karunaratne et al., 2020], language processing [Najafabadi et al., 2016], robotics [Neubert et al., 2019], and graph-structured data [Poduval et al., 2022, Nunes et al., 2022]. However, the ecosystem of open-source VSA tools remains relatively small. The most prominent library, *torchhd* [Heddes et al., 2023], targets GPU-accelerated research workloads. There is a gap for a minimal, pedagogically oriented Python implementation that exposes the core ideas without heavy dependencies.

*PaperWings* fills this gap. It is a pure-Python (NumPy) library that implements the three most common VSA families, wraps them in an associative memory with biologically-motivated forgetting, and provides a triple-unbinding algorithm for knowledge graph recovery. This paper introduces the necessary background, describes the library architecture, and presents worked examples.

Our contributions are: (1) a minimal pedagogical implementation of core VSA operations, (2) an associative memory with explicit exponential decay, and (3) a practical triple-unbinding algorithm for recovering relational structure.

# 2 Background

## 2.1 Foundations of Hyperdimensional Computing

The intellectual roots of VSAs lie in Kanerva's *Sparse Distributed Memory* [Kanerva, 1988], which modelled human long-term memory as a content-addressable store in a high-dimensional binary space. Smolensky [1990] showed that tensor products could encode variable bindings in connectionist networks, and Plate [1995] compressed these representations via circular convolution in his *Holographic Reduced Representations* (HRR). Kanerva [1997] introduced the *Binary Spatter Code* (BSC), using XOR for binding and majority rule for bundling, while Rachkovskij & Kussul [2001] proposed *context-dependent thinning* to bind sparse binary vectors without losing sparsity.

Gayler [2003] unified these approaches under the term *Vector Symbolic Architecture* and demonstrated their capacity for compositional linguistic structure. Kanerva [2009] provided an accessible tutorial that remains the standard entry point, and Levy & Gayler [2008] argued for their relevance to artificial general intelligence.

## 2.2 The VSA Framework

All VSAs share three core operations on vectors of dimensionality $d$ (typically $d \geq 1000$):

1. **Binding** ($\otimes$): Combines two vectors into a new vector that is dissimilar to both operands. Binding is used to represent associations (e.g., role–filler pairs).

2. **Bundling** ($\oplus$): Combines two or more vectors into a new vector that is *similar* to all operands. Bundling creates superpositions—composite representations.

3. **Similarity** ($\delta$): Measures the distance or overlap between two vectors, enabling content-addressable retrieval.

A critical property is that binding is designed to be invertible (exactly in some algebras, approximately in others): given $\mathbf{c} = \mathbf{a} \otimes \mathbf{b}$, one can recover $\mathbf{b} \approx \mathbf{a} \otimes \mathbf{c}$ (exactly for binary XOR; approximately for other algebras). This enables *unbinding*—querying a composite for one of its constituents.

## 2.3 Common VSA Families

Table 1: Common VSA families used in this work.

| **Family** | **Domain** | **Binding** | **Bundling** | **Similarity** |
|---|---|---|---|---|
| BSC | $\{0, 1\}$ | XOR | Majority vote | Norm. Hamming |
| MAP | $\{-1, +1\}$ | Elem. multiply | Elem. add + sign | Norm. dot product |
| SBDR | $\{0, 1\}$, sparse | Permuted OR | Permuted OR | Jaccard distance |

The library implements these three families; HRR (real-valued, circular convolution) is omitted for simplicity.

### 2.4 Related Software

*torchhd* [Heddes et al., 2023] provides a PyTorch-based library supporting BSC, MAP, HRR, and several other families with GPU acceleration. *OpenHD* [Kang et al., 2022] targets GPU-accelerated classification workloads. The *VSA Toolbox* [Schlegel et al., 2022] offers a MATLAB environment for comparative VSA research. The library presented here complements these by prioritising pedagogical clarity and minimal dependencies.

### 2.5 Memory Decay

Ebbinghaus [1885] established that human memory retention follows an approximately exponential decay curve. This phenomenon has motivated extensive work in continual learning, where neural networks must avoid *catastrophic forgetting* when learning sequentially [Kirkpatrick et al., 2017, De Lange et al., 2022]. The library incorporates a simple exponential decay model into its vector memory, providing a mechanism for biologically-inspired forgetting that is uncommon in existing VSA libraries.

### 2.6 Knowledge Graphs and HDC

Encoding graph-structured knowledge in hyperdimensional space has attracted recent attention. Poduval et al. [2022] introduced *GrapHD* for graph memorisation with short- and long-term memory. Nunes et al. [2022] demonstrated graph classification competitive with GNNs at a fraction of the computational cost. Frady et al. [2021] analysed binding capacity for sparse distributed representations with applications to relational data. This work contributes a triple-unbinding algorithm that recovers (subject, predicate, object) triples from bound vectors by enumerating candidate combinations with hierarchical pruning.

## 3 Library Design

The library is organised into four modules:

```
paperwings/
+-- vector/
|   +-- vector.py          # Vector types and operations
|   +-- vector_space.py    # Associative memory
+-- unbinder/
|   +-- triple_unbinder.py # Knowledge graph recovery
+-- exceptions/
|   +-- memory_exception.py
+-- util/
    +-- logging_util.py
    +-- file_path_util.py
```

The only external dependency is NumPy. Metadata is persisted in SQLite (standard library).

### 3.1 Vector Types

All vector types inherit from `AbstractVector`, which defines the interface for `add` (bundling), `mul` (binding), `sub` (subtraction), and `dist` (similarity), with operator overloads `+`, `*`, `-`. A factory method creates vectors of any type:

```python
from paperwings.vector.vector import AbstractVector

v = AbstractVector.new_vector(size=1000, rep="binary")
```

**BinaryVector (BSC).** Elements are drawn uniformly from $\{0, 1\}$.

- *Binding:* $\mathbf{z} = \mathrm{XOR}(\mathbf{x}, \mathbf{y})$. Self-inverse: $\mathrm{XOR}(\mathrm{XOR}(\mathbf{a}, \mathbf{b}), \mathbf{a}) = \mathbf{b}$.
- *Bundling:* Element-wise sum followed by majority rule. For two vectors, ties ($x_i + y_i = 1$) are broken randomly; for more than two vectors, the majority value is selected.
- *Distance:* Normalised Hamming distance: $\delta(\mathbf{x}, \mathbf{y}) = \frac{1}{d} \sum_i \mathrm{XOR}(x_i, y_i)$.

**BipolarVector (MAP).** Elements are drawn uniformly from $\{-1, +1\}$.

- *Binding:* Element-wise multiplication. Self-inverse: $(\mathbf{a} \cdot \mathbf{b}) \cdot \mathbf{a} = \mathbf{b}$.
- *Bundling:* Element-wise addition followed by sign thresholding to $\{-1, +1\}$; ties randomised.
- *Distance:* $\delta(\mathbf{x}, \mathbf{y}) = \frac{d - \mathbf{x}^\top \mathbf{y}}{2d}$.

**BinarySparseVector (SBDR).** Elements are drawn from $\{0, 1\}$ with sparsity $s = 0.2$.

- *Binding and Bundling:* Permuted OR with $k = 8$ random permutations, followed by AND with the initial OR. This preserves sparsity.
- *Distance:* Jaccard distance: $\delta(\mathbf{x}, \mathbf{y}) = 1 - \frac{\sum_i \mathrm{AND}(x_i, y_i)}{\sum_i \mathrm{OR}(x_i, y_i)}$.

### 3.2 Vector Space (Associative Memory)

`VectorSpace` manages a named collection of vectors and provides content-addressable retrieval:

```python
from paperwings.vector.vector_space import VectorSpace

space = VectorSpace(size=1000, rep="binary")
country = space.add_vector("COUNTRY")
usa     = space.add_vector("USA")
dollar  = space.add_vector("DOLLAR")
```

Key methods:

- `add_vector(name)` — create and store a new random vector.
- `insert_vector(v, name)` — store a pre-existing vector.
- `find_vector(x)` — return `(name, distance)` of the nearest stored vector.
- `space["USA"]` — retrieve a vector by name.
- `delete_vector(name)` — remove a vector.

### 3.3 Memory Decay

The library models forgetting with exponential decay inspired by the Ebbinghaus curve:

$$\text{strength}(t) = \text{strength}_0 \cdot e^{-\lambda t} \tag{1}$$

Each vector carries a `strength` attribute (initially 100). Calling `space.decay(decay_rate=0.05, time_passed=5)` updates all vectors; those whose strength falls below a configurable threshold (0.5 by default) are deleted. A typical trajectory is shown in Table 2.

Table 2: Memory strength over repeated decay calls ($\lambda = 0.05$, $\Delta t = 5$).

| **Decay calls** | **Effective $t$** | **Strength** |
|---|---|---|
| 0 | 0 | 100.0 |
| 1 | 5 | 77.9 |
| 2 | 10 | 60.7 |
| 5 | 25 | 28.7 |
| 10 | 50 | 8.2 |
| $\sim$15 | $\sim$75 | $\approx 0.5$ (deleted) |

### 3.4 Triple Unbinder

The `TripleUnbinder` recovers (subject, predicate, object) triples from a bound vector by enumerating candidate combinations. Given $\mathbf{b} = \mathbf{s} \otimes \mathbf{p} \otimes \mathbf{o}$:

```
from paperwings.unbinder.triple_unbinder import TripleUnbinder

unbinder = TripleUnbinder(space, early_stop=True, top_k=20)
result = unbinder.unbind(bound_vector)
# -> ("Socrates", "is_a", "man")
```

**Algorithm.**

1. *Pre-filter:* Compute cosine similarity between $\mathbf{b}$ and every stored vector; retain the top-$k$ candidates.
2. *Enumerate:* For each combination of three candidates $(\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3)$, compute $\mathbf{v}_1 \otimes \mathbf{v}_2 \otimes \mathbf{v}_3$ and measure its Hamming distance to $\mathbf{b}$.
3. *Select:* Return the triplet with minimum distance. If `early_stop=True` and an exact match (distance $= 0$) is found, return immediately.
4. *Parallelise (optional):* When $k > 20$, triplet evaluation may be distributed across threads via `ThreadPoolExecutor`.

Cosine similarity is used for candidate pre-filtering, while Hamming distance is used for exact comparison. The current triple-unbinding implementation targets binary and binary-sparse representations, for which Hamming-style exact comparison is natural.
Complexity is $O(nd + n \log n + k^3 d)$, where $n$ is the vocabulary size, $k$ is the candidate count, and $d$ is the dimensionality. For the default $k = 20$, this yields $\binom{20}{3} = 1{,}140$ triplet evaluations—tractable for interactive use.

# 4 Tutorial: Worked Examples

## 4.1 Encoding Semantic Facts

We encode two facts: "The USA uses the Dollar" and "Mexico uses the Peso."

```
from paperwings.vector.vector_space import VectorSpace

space = VectorSpace(size=1000, rep="binary")

# Role vectors
COUNTRY  = space.add_vector("COUNTRY")
CURRENCY = space.add_vector("CURRENCY")

# Filler vectors
USA    = space.add_vector("USA")
DOLLAR = space.add_vector("DOLLAR")
MEXICO = space.add_vector("MEXICO")
PESO   = space.add_vector("PESO")

# Encode facts as role-filler bundles
usa_record    = (COUNTRY * USA) + (CURRENCY * DOLLAR)
mexico_record = (COUNTRY * MEXICO) + (CURRENCY * PESO)

space.insert_vector(usa_record, "USA_RECORD")
space.insert_vector(mexico_record, "MEXICO_RECORD")
```

Listing 1: Encoding semantic facts with role–filler binding.

**Querying.** To ask "What currency does the USA use?", we unbind the CURRENCY role from the USA record:

```
query = usa_record * CURRENCY  # unbind CURRENCY role
name, dist = space.find_vector(query)
print(name, dist)  # -> "DOLLAR", ~0.0
```

Listing 2: Querying by unbinding.

## 4.2 Knowledge Graph Triples

We encode an ontological triple and recover it:

```
from paperwings.vector.vector_space import VectorSpace
from paperwings.unbinder.triple_unbinder import TripleUnbinder

space = VectorSpace(size=1000, rep="binary_sparse")

socrates = space.add_vector("Socrates")
is_a     = space.add_vector("is_a")
man      = space.add_vector("man")
mortal   = space.add_vector("mortal")

# Encode triple: Socrates is_a man
triple = socrates * is_a * man

```

```python
# Recover the triple
unbinder = TripleUnbinder(space, early_stop=True, top_k=10)
result = unbinder.unbind(triple)
print(set(result))  # -> {"Socrates", "is_a", "man"}
```

Listing 3: Triple encoding and recovery.

### 4.3 Memory Decay

We demonstrate how memories fade over simulated time:

```python
space = VectorSpace(size=1000, rep="binary")
v = space.add_vector("ephemeral")

print(f"Initial strength: {space['ephemeral'].strength}")

for i in range(1, 16):
    space.decay(decay_rate=0.05, time_passed=5)
    if "ephemeral" in space.vectors:
        print(f"After {i} steps: "
              f"{space['ephemeral'].strength:.1f}")
    else:
        print(f"After {i} steps: vector forgotten")
        break
```

Listing 4: Exponential memory decay.

### 4.4 Comparing Vector Types

The three representations offer different trade-offs:

```python
for rep in ["binary", "bipolar", "binary_sparse"]:
    s = VectorSpace(size=1000, rep=rep)
    a = s.add_vector("a")
    b = s.add_vector("b")

    bound = a * b
    recovered = bound * a  # unbind
    name, dist = s.find_vector(recovered)
    print(f"{rep:15s}  recovered={name}  "
          f"distance={dist:.4f}")
```

Listing 5: Binding and unbinding across vector types.

Binary and bipolar vectors support exact self-inverse unbinding (distance $\approx 0$). Binary sparse vectors use a different binding mechanism and typically require the triple unbinder for reliable recovery.

## 5 Discussion

**Strengths.** PaperWings provides a minimal, dependency-light implementation that makes VSA concepts tangible. The inclusion of memory decay and triple unbinding extends standard VSA implementations and connects to active research on continual learning and knowledge representation.

**Limitations.** The library is a research prototype. The linear-scan retrieval in `find_vector` is $O(nd)$ and does not scale to large vocabularies; approximate nearest-neighbour indices would be needed. The triple unbinder's $O(k^3)$ enumeration is practical for small $k$ but does not generalise to higher-arity relations. GPU acceleration is not supported.

**Future directions.** Possible extensions include (1) approximate nearest-neighbour retrieval (e.g., via locality-sensitive hashing, which is particularly natural in binary spaces), (2) support for sequence encoding via permutation, (3) integration with embedding models to initialise vectors from pre-trained representations, and (4) benchmarking against *torchhd* on standard HDC tasks.

## 6 Conclusion

This paper presents PaperWings, a pedagogical Python library for Vector Symbolic Architectures. By implementing three vector families, an associative memory with biologically-inspired decay, and a triple-unbinding algorithm for knowledge graph recovery, the library provides a self-contained environment for learning and experimenting with hyperdimensional computing. We hope it lowers the barrier to entry for researchers and students exploring this brain-inspired computational paradigm.

## References


De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., Slabaugh, G., & Tuytelaars, T. (2022). A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7), 3366–3385.

Ebbinghaus, H. (1885). *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie.* Leipzig: Duncker & Humblot.

Frady, E. P., Kleyko, D., & Sommer, F. T. (2021). Variable binding for sparse distributed representations: Theory and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(3), 1251–1262.

Gayler, R. W. (2003). Vector symbolic architectures answer Jackendoff's challenges for cognitive neuroscience. In *Proceedings of the ICCS/ASCS Joint International Conference on Cognitive Science*, 133–138.

Heddes, M., Nunes, I., Vergés, P., Kleyko, D., Abraham, D., Givargis, T., Nicolau, A., & Veidenbaum, A. (2023). Torchhd: An open source Python library to support research on hyperdimensional computing and vector symbolic architectures. *Journal of Machine Learning Research*, 24(255), 1–10.

Kang, J., Khaleghi, B., Rosing, T., & Kim, Y. (2022). OpenHD: A GPU-powered framework for hyperdimensional computing. *IEEE Transactions on Computers*, 71(11), 2753–2764.

Kanerva, P. (1988). *Sparse Distributed Memory.* MIT Press.

Kanerva, P. (1997). Fully distributed representation. In *Proceedings of the Real World Computing Symposium (RWC'97)*, 358–365.

Kanerva, P. (2009). Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 1(2), 139–159.

Karunaratne, G., Le Gallo, M., Cherubini, G., et al. (2020). In-memory hyperdimensional computing. *Nature Electronics*, 3, 327–337.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526.

Levy, S. D. & Gayler, R. W. (2008). Vector symbolic architectures: A new building material for artificial general intelligence. In *Proceedings of the First Conference on Artificial General Intelligence (AGI 2008)*, IOS Press.

Najafabadi, F. R., Rahimi, A., Kanerva, P., & Rabaey, J. M. (2016). Hyperdimensional computing for text classification. In *Proceedings of the Design, Automation & Test in Europe Conference (DATE 2016)*.

Neubert, P., Schubert, S., & Protzel, P. (2019). An introduction to hyperdimensional computing for robotics. *KI – Künstliche Intelligenz*, 33(4), 319–330.

Nunes, I., Heddes, M., Givargis, T., Nicolau, A., & Veidenbaum, A. (2022). GraphHD: Efficient graph classification using hyperdimensional computing. arXiv:2205.07826.

Plate, T. A. (1995). Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3), 623–641.

Poduval, P., Alimohamadi, H., Zakeri, A., Imani, F., Najafi, M. H., Givargis, T., & Imani, M. (2022). GrapHD: Graph-based hyperdimensional memorization for brain-like cognitive learning. *Frontiers in Neuroscience*, 16, 757125.

Rachkovskij, D. A. & Kussul, E. M. (2001). Binding and normalization of binary sparse distributed representations by context-dependent thinning. *Neural Computation*, 13(2), 411–452.

Rahimi, A., Kanerva, P., & Rabaey, J. M. (2016). A robust and energy-efficient classifier using brain-inspired hyperdimensional computing. In *Proceedings of the International Symposium on Low Power Electronics and Design (ISLPED '16)*, ACM.

Schlegel, K., Neubert, P., & Protzel, P. (2022). A comparison of vector symbolic architectures. *Artificial Intelligence Review*, 55, 4523–4555.

Smolensky, P. (1990). Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artificial Intelligence*, 46(1–2), 159–216.

Thomas, A., Dasgupta, S., & Rosing, T. (2021). A theoretical perspective on hyperdimensional computing. *Journal of Artificial Intelligence Research*, 72, 215–249.